Book Finder

Introduction:

Book Finder is an exclusive app designed for BookRecommenders to curate and suggest books tailored to readers' preferences. It provides tools for BookRecommenders to create and manage book lists, track trending genres, and select books that align with diverse reading interests. By leveraging Book Finder, BookRecommenders can craft a more engaging, personalized reading experience for their audiences.

User of the System:

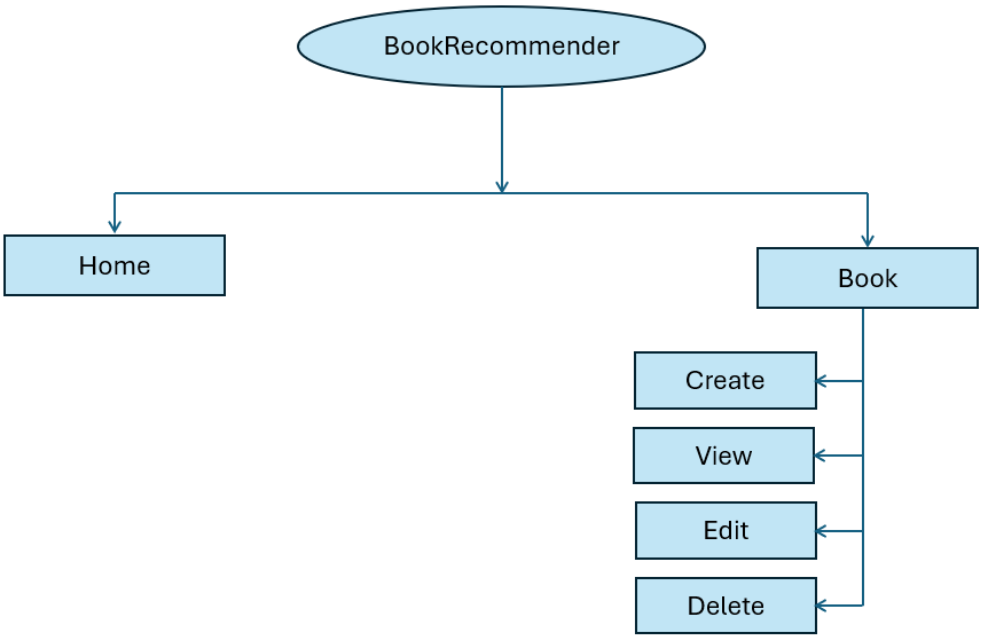
- BookRecommender - The primary user responsible for curating book lists, managing genres, and recommending trending books.

Modules of the Application:

BookRecommender:

- Home
  - Add Book
  - View Book Details
  - Edit Book
  - Delete Book

BookRecommender Flow Diagram:



Technology Stack

Front End
React
Back End
Springboot, Microsoft SQL Server Database.

BookController.java

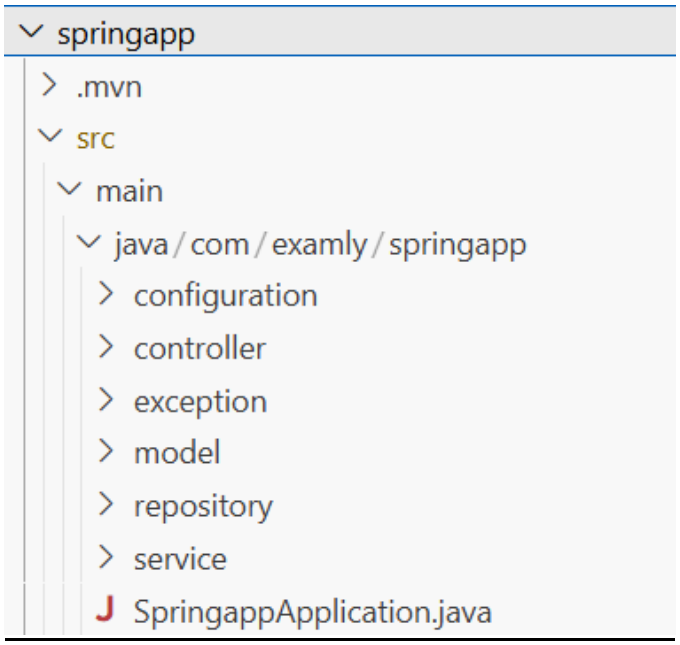
Action	URL	HTTP Method	Response
Add a new book	/api/books	POST	201 Created if successful, 400 Bad Request if failure
Get all books	/api/books	GET	200 OK with a list of books, 500 Internal Server Error if not
Edit an existing book	/api/books/{bookId}	PUT	201 Created if successful, 404 Not Found if book doesn't exist
Get a book by ID	/api/books/{bookId}	GET	200 OK with the book, 404 Not Found if book doesn't exist
Delete a book	/api/books/{bookId}	DELETE	200 OK if successful, 404 Not Found if book doesn't exist

The **BookController** class serves as the REST API controller for managing **Book** entities, delegating operations to **BookService** methods. It provides HTTP endpoints for basic CRUD operations and wraps responses in **ResponseEntity** for status handling.

- **Get All Books (@GetMapping):** Retrieves and returns all books as a list in the response body with a **200 OK** status.
- **Get Book by ID (@GetMapping("/{bookId}")):** Fetches a specific book by its ID. If found, the book is returned with a **200 OK** status and includes the book's details in the response. If not, a **404 Not Found** status is returned with the message "**Book not found**".
- **Add New Book (@PostMapping):** Adds a new book to the repository. If successful, the book is returned with a **201 Created** status. If a duplicate title exists, a **400 Bad Request** status is returned with the message "**Failed to add book**".
- **Update Book (@PutMapping("/{bookId}")):** Updates an existing book by ID. If the book is successfully updated, it is returned with a **201 Created** status and includes the updated details in the response. If the book is not found, a **404 Not Found** status is returned with the message "**Failed to update book**".
- **Delete Book (@DeleteMapping("/{bookId}")):** Deletes a specific book by ID. If the deletion is successful, a **200 OK** status is returned with the message "**Book deleted successfully**". If the book is not found, a **404 Not Found** status is returned with the message "**Failed to delete book**".

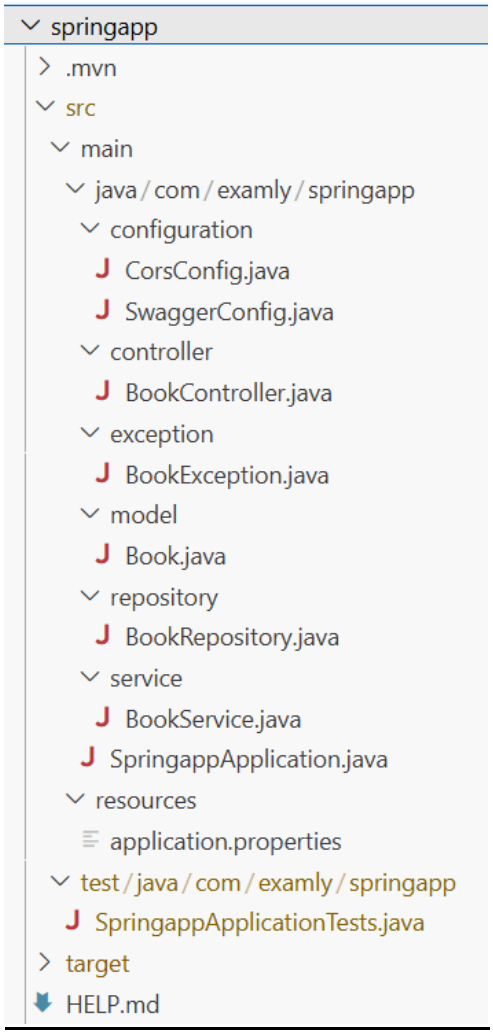
Backend Requirements:

Create folders named as model, controller, service, exception and configuration inside **springapp** of the src folder.



- In **configuration** folder, create a class **CorsConfig** with **@Configuration** annotation which implements **WebMvcConfigurer** to enable CORS functionality and create a **SwaggerConfig** class configures Swagger for the Spring Boot application, enabling automatic generation of API documentation.
- In **controller** folder, create class named **BookController** to implement REST APIs (Refer the API table).
- In **repository** folder, create the interfaces named as **BookRepository** which extends **JpaRepository**.
- In **service** folder, create a class **BookService** that provides the core business logic for handling books.
- In **exception** folder, create a class **BookException** is a custom exception that extends Exception and provides a way to handle specific error messages related to book operations.
- Ensure adherence to the specified API outlined in the API table, ensuring consistency in class names, variables, and folder structure as detailed.

Backend folder structure:



<u>Model class:</u>

Book.java - This class represents a book entity, with fields to store details such as the book's title, author, genre, publication date, and cover image.

Properties:

- **bookId: int**
  - Annotation: @Id
  - Generation Strategy: @GeneratedValue(strategy = GenerationType.IDENTITY)
- **title: String**
  - Description: The title of the book.
- **author: String**
  - Description: The author of the book.
- **genre: String**
  - Description: The genre or category of the book.
- **publishedDate: String**
  - Description: The publication date of the book, formatted as a string.
- **coverImage: String**
  - Annotation: @Lob, @Column(name = "image", nullable = false, columnDefinition = "LONGBLOB")

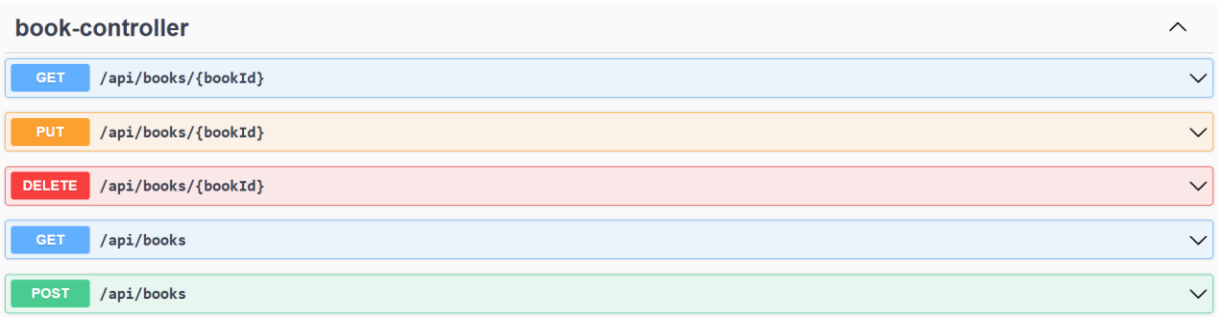
- Description: The cover image of the book stored as a large object (LOB) in the database.

BookService.java:

The **BookService** class serves as a service layer to manage CRUD operations on Book entities, using **BookRepository** to interact with the database. This class handles various operations and provides meaningful error messages when exceptions occur:

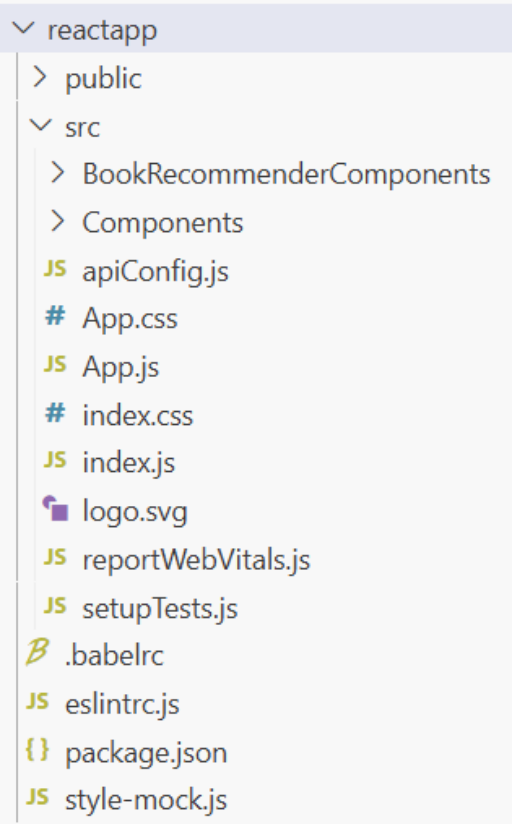
- **Get All Books (getAllBooks):** Retrieves and returns a list of all books from the repository.
- **Get Book by ID (getBookById):** Fetches a book by its ID, returning the book if found. If not, it throws a **BookException** with the message **"Book not found with ID: {bookId}"**.
- **Add New Book (addBook):** Adds a new book to the repository. Before saving, it checks if a book with the same title already exists. If a duplicate title is found, it throws a **BookException** with the message **"Book with title '{book.getTitle()}' already exists"**. If no duplicate is found, the book is saved to the repository.
- **Update Book (updateBook):** Updates an existing book based on its ID. It first retrieves the book using its ID; if the book is found, all fields (title, author, genre, published date, and cover image) are updated with values from updatedBook. If the book is not found, a **BookException** is thrown with the message **"Cannot update. Book not found with ID: {bookId}"**.
- **Delete Book (deleteBook):** Deletes a book by its ID. If a book with the specified ID exists, it is deleted from the repository. If the book is not found, a **BookException** is thrown with the message **"Cannot delete. Book not found with ID: {bookId}"**.

Endpoints:

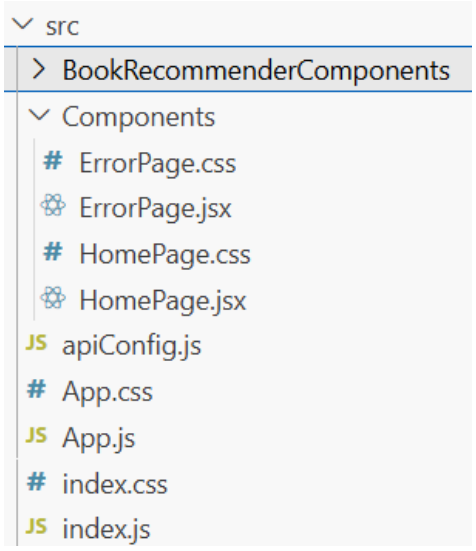


Frontend Requirements:

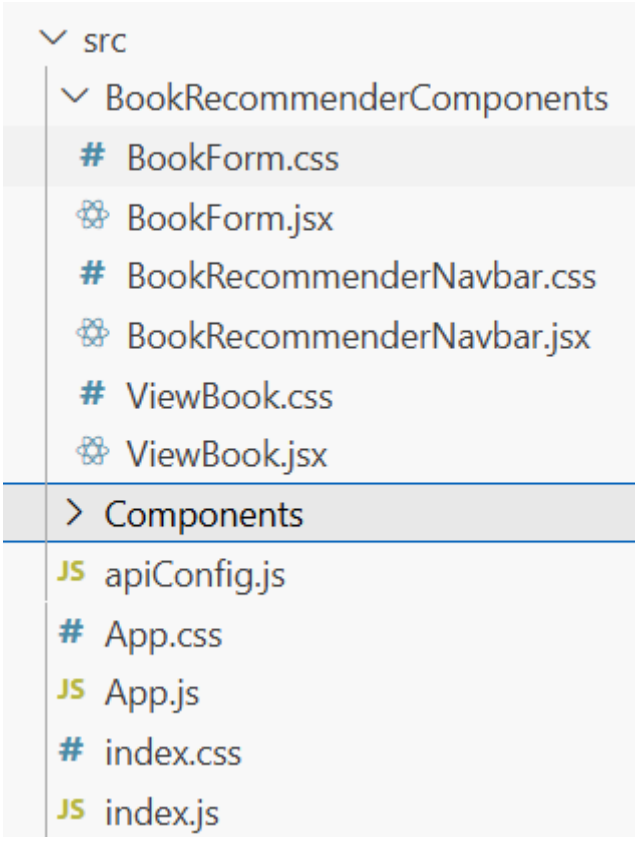
Book Folder Screenshot:



Components:



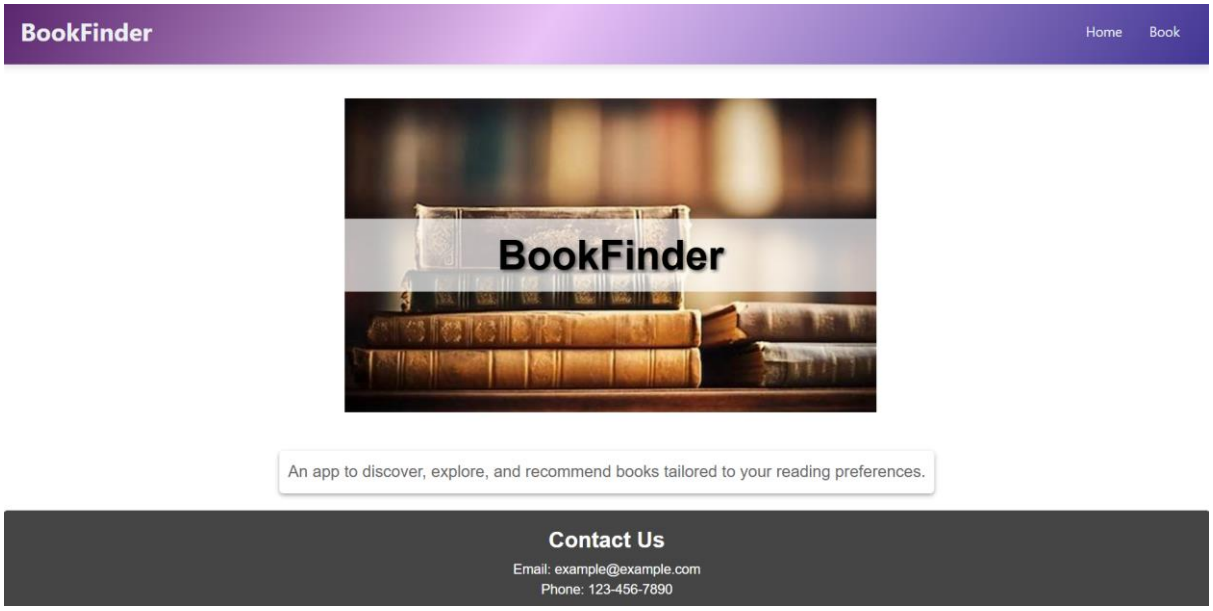
BookRecommenderComponents:



Frontend Screenshots:

• All the asterisk (*) marked fields are mandatory in the form. Make sure to mark all the field names with * symbol followed by the validations.

Landing page:



App.js :

The App component in React serves as the main routing setup for the Book Finder application, using **react-router-dom** to manage different page routes and handle navigation between them. Here's a detailed description of each route and component:

Router Setup:

The app uses **BrowserRouter** as Router to enable routing across multiple pages within the app.

Routes:

- **ErrorPage (/error):** Displays an error page when an invalid route is accessed or if a route fails to load correctly.
- **HomePage (/home):** Serves as the main landing page where users can view an introduction and details about Book Finder.
- **BookForm (/newbook):** This route allows users to access a form for adding new book recommendations.
- **EditBook (/editbook/:id):** Similar to BookForm, this route is used to edit an existing book based on its unique id, passed as a URL parameter.
- **ViewBook (/viewbook):** Takes users to a page where they can view individual book details.

Default Redirect to Home:

The root path (/) automatically redirects users to the **HomePage** route at **/home**, ensuring a consistent entry point.

Catch-all Redirect to Error:

Any undefined paths (using *) redirect to **/error**, guiding users to a designated error page if they enter an unrecognized URL.

HomePage Component:

The HomePage component in React serves as the main interface for the Book Finder app, featuring a navigation bar, a cover image with a title overlay, a description, and contact information. The structure of the component covers all elements :

Navigation Bar:

BookRecommenderNavbar is included at the top of the page, providing navigation links. This is rendered within a **<nav>** element, giving it the role of navigation to meet accessibility requirements.

Cover Image and Title:

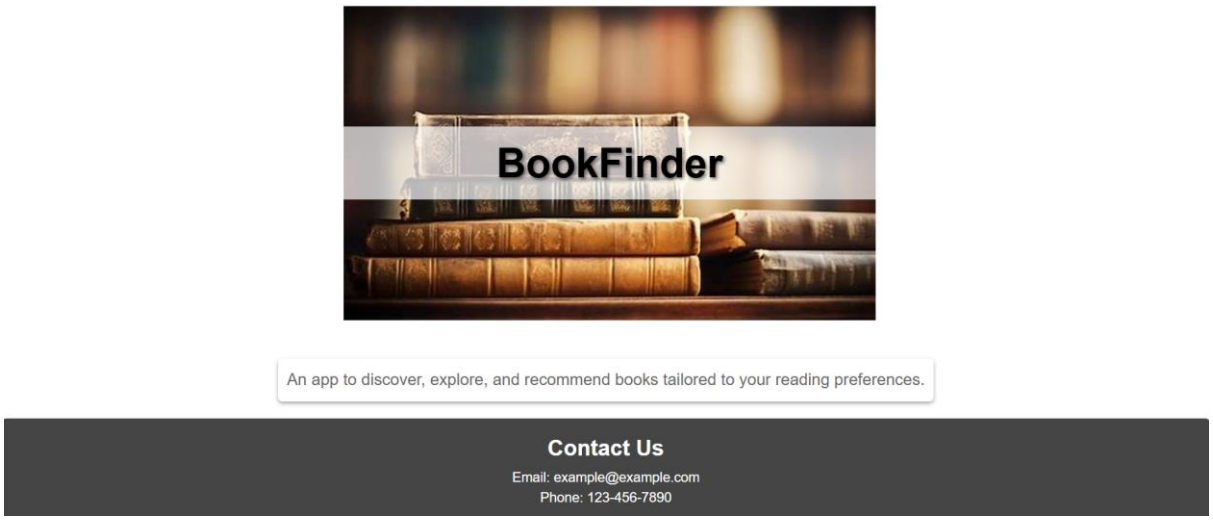
- The component displays a cover image using **LazyLoadImage** for optimized loading with a blur effect.
- The image file, **bookfindercoverimage.jpeg**, is accessed from the public folder.
- A title, "**BookFinder**" is overlaid on the cover image and is contained within a div with the class title.

App Description:

Below the cover image, a paragraph briefly describes the app: "**An app to discover, explore, and recommend books tailored to your reading preferences.**"

Contact Information:

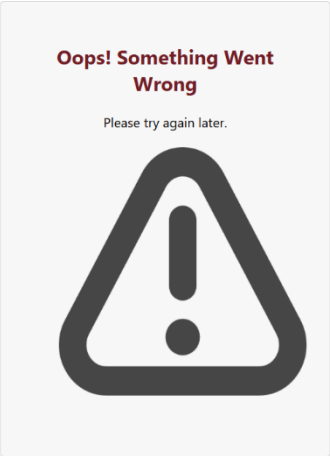
A "**Contact Us**" section displays an email (**example@example.com**) and a phone number (**123-456-7890**), providing contact details.



<u>ErrorPage Component:</u>

Create custom error pages for different HTTP status codes (e.g., 404 Not Found, 500 Internal Server Error) to provide a consistent and user-friendly error experience. Ensure that error pages contain helpful information and guidance for users.

Thus, create a reliable and user-friendly web application that not only meets user expectations but also provides a robust and secure experience, even when faced with unexpected situations. Error page must be displayed if something goes wrong.



BookRecommenderComponents:

BookRecommenderNavbar Component:

The BookRecommenderNavbar component is a navigation bar designed for the Book Finder app, providing users with easy access to key functionalities. Below are its main features:

Site Title:

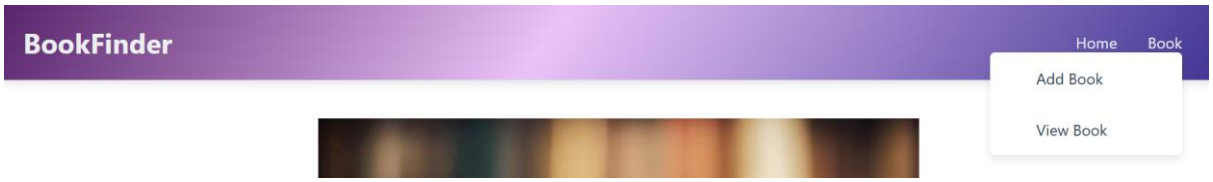
- The title **BookFinder** is prominently displayed at the top of the navigation bar.
- It serves as a clickable link that directs users to the **/home** route, ensuring quick access to the main interface of the application.

Navigation Links:

Home Link: This link, which also points to the **/home** route, allows users to return to the Home page directly.

Dropdown Menu: Labeled **Book**, this menu contains additional options related to book management:

- **Add Book:** This link routes to **/newbook**, allowing users to access a form where they can add new book entries to the app.
- **View Book:** This link navigates to **/viewbook**, where users can browse or search for existing books within the application.



<u>BookForm component:</u>

The BookForm component is designed for creating or editing book entries in a book recommendation application. It includes form fields for entering book details, uploading a cover image, and provides user feedback through success messages and error handling.

Use a conditional (ternary) operator to display a different heading based on the presence of an **id** value:

- If **id** is present (meaning an existing book is being edited), it displays "**Edit Book**".
- If **id** is absent (indicating a new book is being created), it displays "**Create New Book**".

API Interaction:

- The component interacts with the backend API to fetch book data when editing an entry, and to either create a new entry or update an existing one upon form submission.
- The form uses **axios** for API requests, sending book details (title, author, published date, genre, and cover image) to the backend. This ensures that book information is added to or updated in the application's database.

Fields

Title:

- Id: title
- Input Type: Text
- Placeholder: "Title"

- Validation: Required. Displays error message if left empty.
- Error Message: "**Title is required**"

Author:

- Id: author
- Input Type: Text
- Placeholder: "Author"
- Validation: Required. Displays error message if left empty.
- Error Message: "**Author is required**"

Published Date:

- Id: publishedDate
- Input Type: Date
- Value Format: YYYY-MM-DD
- Validation: Required. Displays error message if left empty.
- Max Date: Today's date to ensure valid date entry.
- Error Message: "**Published Date is required**"

Genre:

- Id: genre
- Input Type: Text
- Placeholder: "Genre"
- Validation: Required. Displays error message if left empty.
- Error Message: "**Genre is required**"

Cover Image:

- Id: coverImage
- Input Type: File (accepts .jpg, .jpeg, .png formats)
- Validation: Required only when creating a new book (not in edit mode). Displays error message if left empty during creation.
- Error Message: "**Cover image is required**"
- Preview: Displays a preview of the uploaded cover image.

Buttons

Back Button:

- Label: "**Back**"
- Functionality: Navigates back to the previous page when clicked.

Submit Button:

- Label: "**Add Book**" (for new entries) or "**Update Book**" (for existing entries)
- Functionality: Submits the form data. If all validations pass, it will add or update the book in the database.

Navigation

- The component uses **useNavigate** from react-router-dom to manage navigation.
- Clicking the **Back** Button navigates to the previous page.
- After successfully adding or updating a book, the user is redirected to the **/viewbook** route by clicking the **Close** button in the success modal.

Success Messages

Upon successful submission:

- Message for Adding a Book: "**Book added successfully!**"
- Message for Updating a Book: "**Book updated successfully!"**

These messages are displayed in a modal overlay with a **Close** button that navigates the user to the book viewing page.

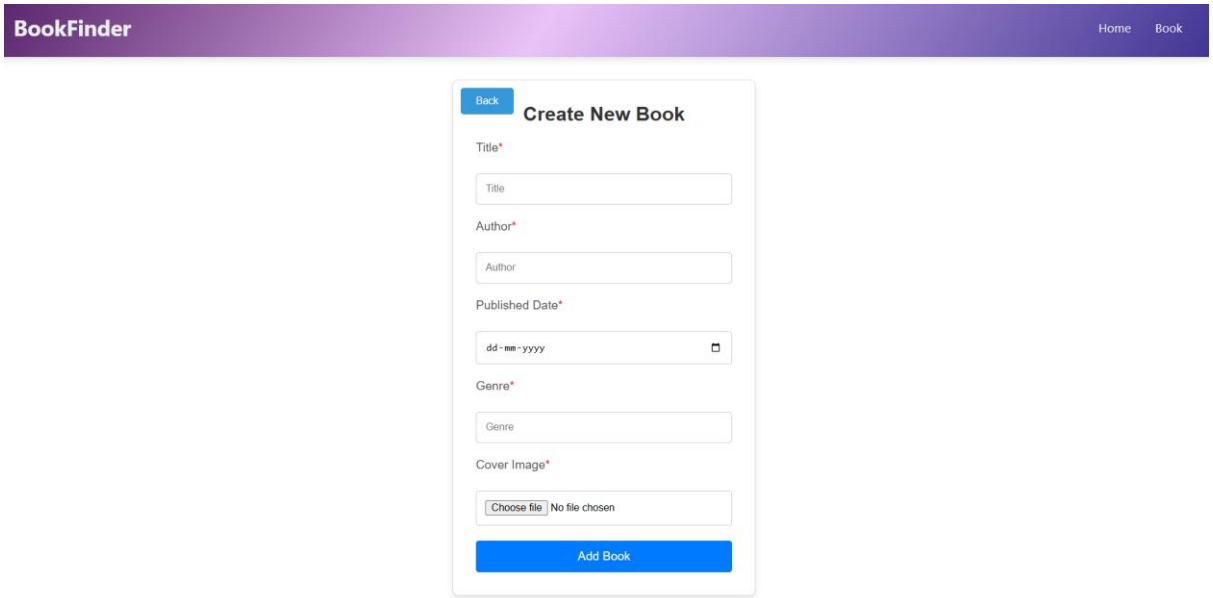
Error Handling

- Each field includes error messages that display below the respective input if validation fails.
- The error messages provide clear guidance on what is required to successfully submit the form.

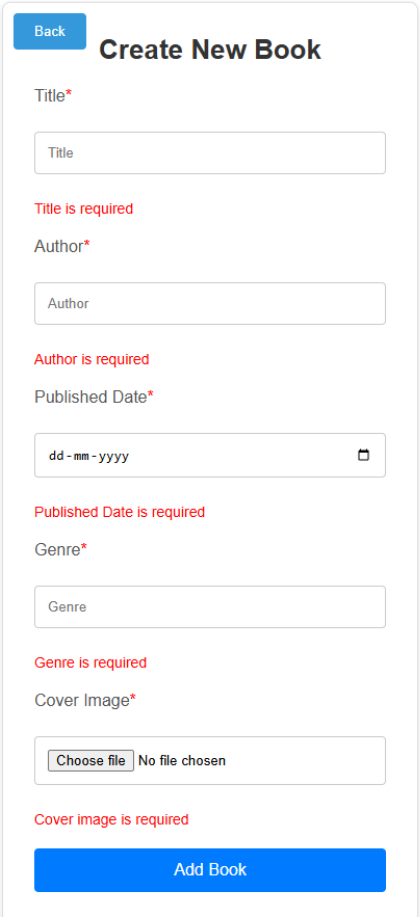
Modal

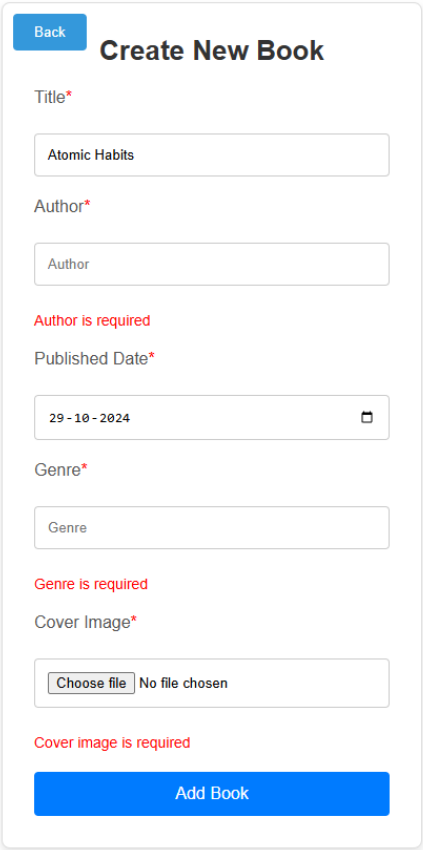
The component displays a modal overlay after a successful submission, providing feedback about the operation's success. It contains the success message and a button to close the modal.

Adding a Book Screenshot:

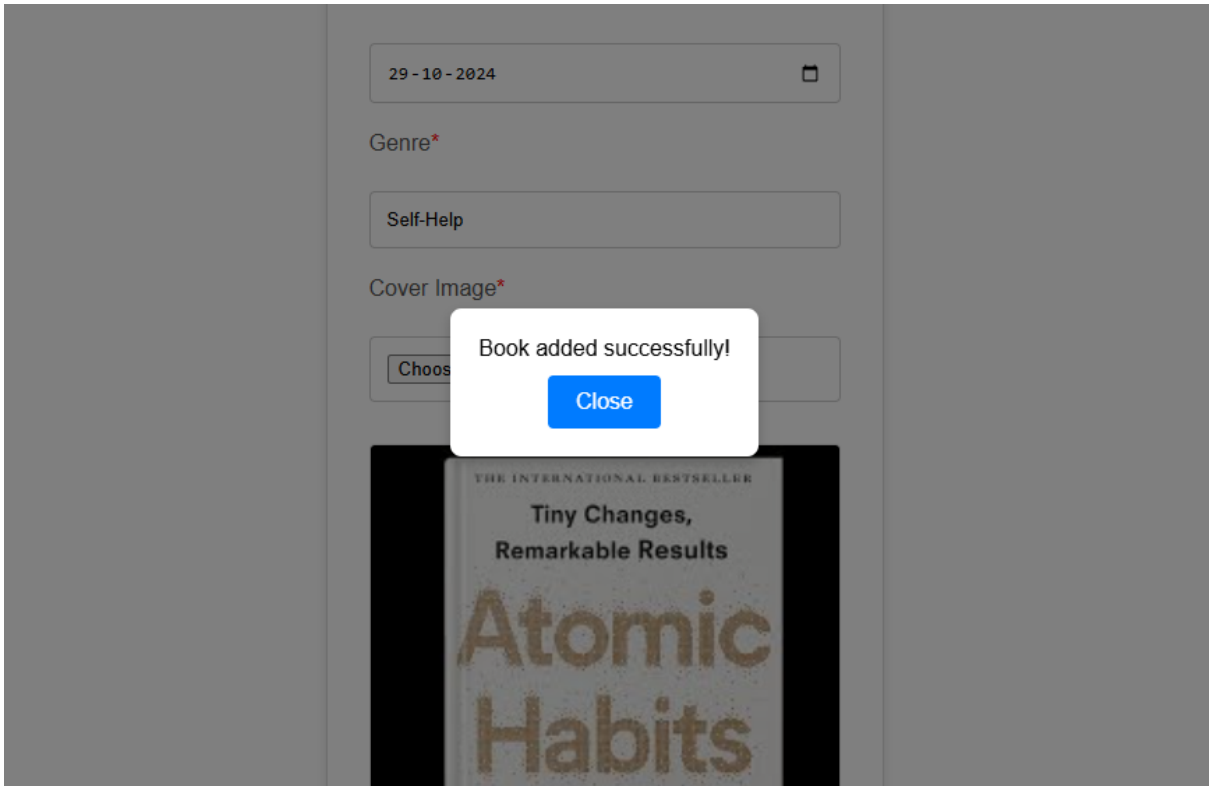


Error messages while adding with empty fields:

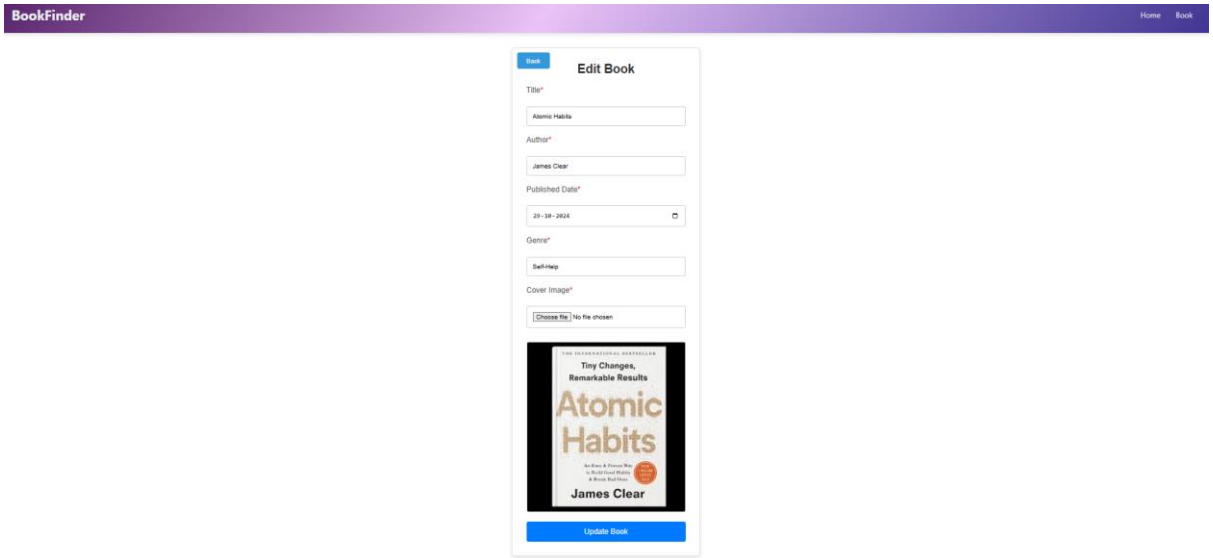




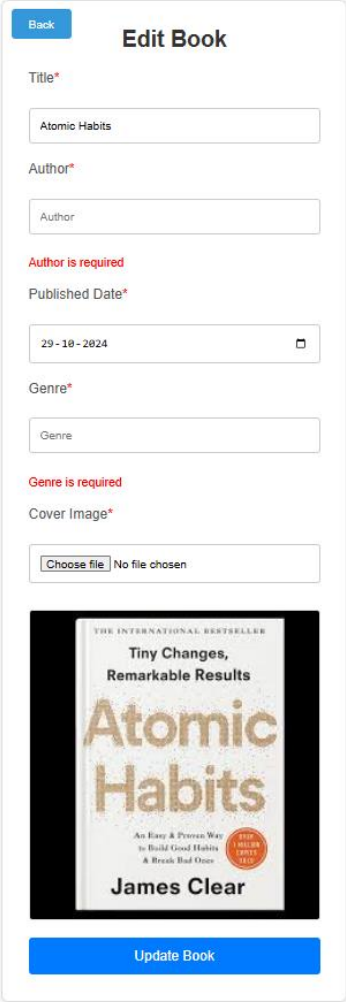
Displaying success message:



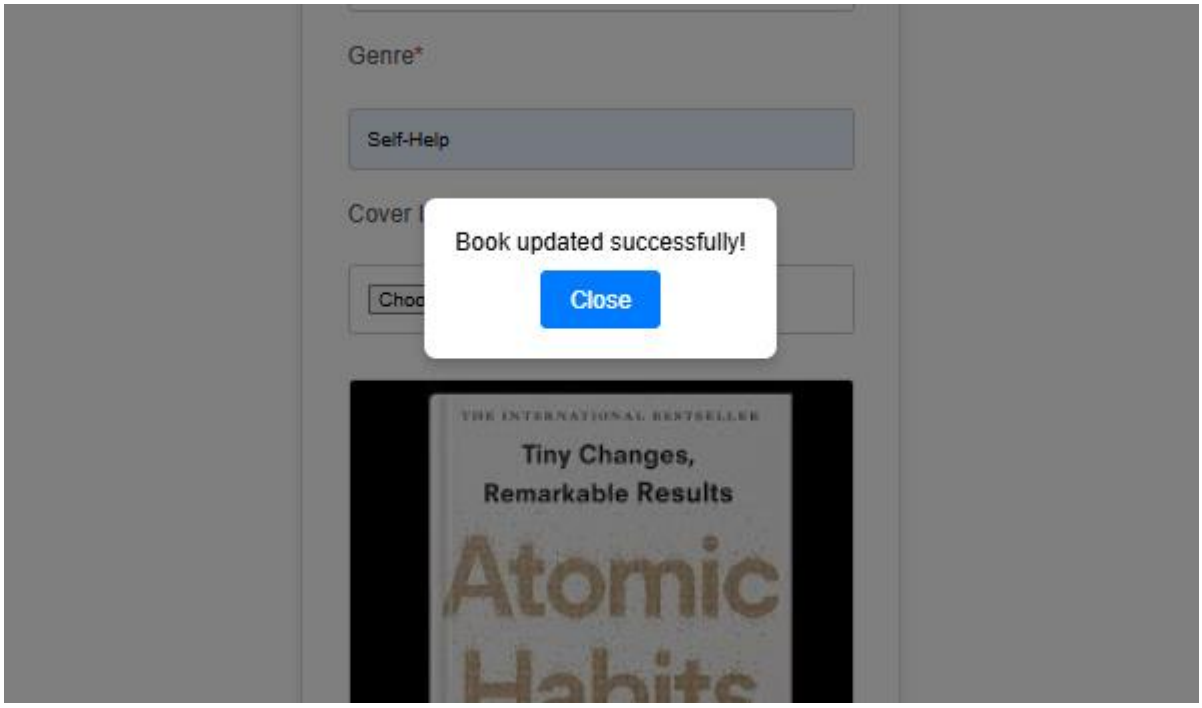
Updating a Book Screenshots:



Error messages while updating with empty fields:



Displaying success message:



<u>ViewBook component:</u>

The ViewBook component is a key part of the Book Recommender application, providing an intuitive interface for users to manage their collection of books. Below are the primary features and functionalities offered by this component:

Books Header:

The component features a prominent header <h1>**Books**</h1> that clearly indicates the section where users can view their book listings.

Book Table:

The core of the component is a structured table that displays detailed information about each book in a user-friendly format. The table consists of the following columns:

- **Cover Image**: Displays the cover image of the book, enhancing the visual appeal and allowing users to quickly identify each title.
- **Title**: Shows the title of the book, making it easy for users to see the names of the books in their collection.
- **Author**: Lists the author of each book, providing context and helping users find works by their favourite writers.

- **Publication Date**: Indicates when the book was published, formatted as "**YYYY-MM-DD**" for consistency and ease of understanding.
- **Genre**: Displays the genre of the book, helping users categorize and filter their reading preferences.
- **Action**: Provides action buttons for each book, allowing users to edit or delete entries directly from the table.

Edit and Delete Actions:

Each row in the table includes buttons for Edit and Delete:

- **Edit**: When clicked, this button navigates users to the edit page for the corresponding book, enabling them to modify its details.
- **Delete**: This button triggers a confirmation popup with the message **"Are you sure you want to delete this book?"** to ensure users intend to remove the book from their collection.
- **Buttons:**
    - **Yes, Delete**: Clicking this button calls the handleConfirmDelete function, which deletes the selected book and refreshes the book list.
    - **Cancel**: This button calls closeDeletePopup, which simply closes the popup without making any changes.

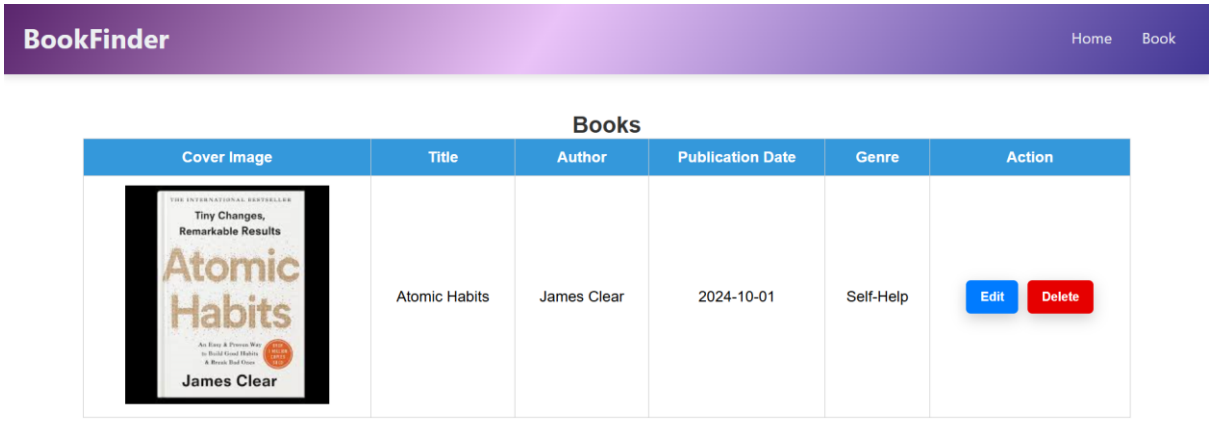
No Records Message:

If there are no books available, the component displays a message within the table indicating "**Oops! No records found**" guiding users on the current state of their book collection.

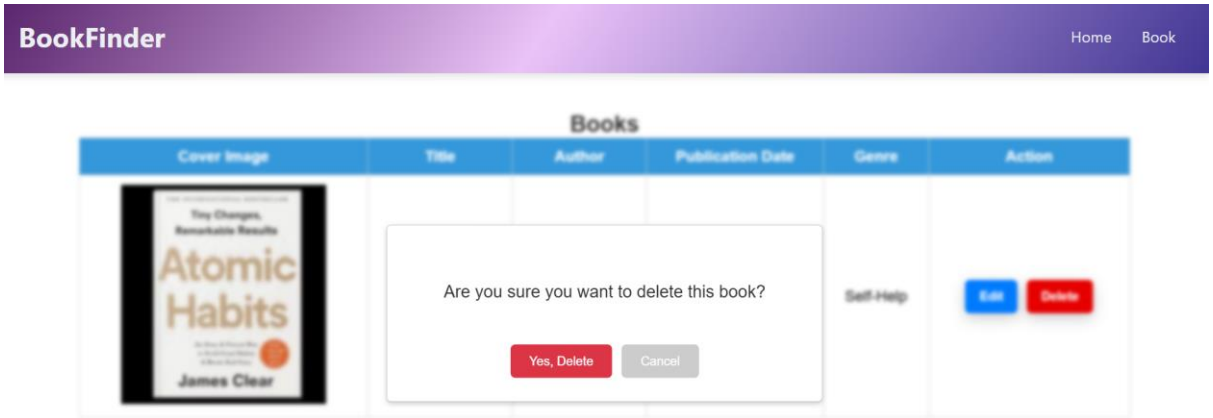
Data Management:

The component fetches the list of available books from the API upon mounting, ensuring that the displayed information is up to date. It handles potential errors gracefully, providing feedback to users if any issues arise during data retrieval.

ViewBook screenshot:



Upon clicking 'Delete':



If no books are found:

Commands to run the project:

BACKEND:

Open the terminal and follow the commands below.

- **cd springapp**

To run the application in port 8080

- **mvn spring-boot: run**

If the same error persists clean the project and build again

To Work with MysqlServer:

(Open a New Terminal) type the below commands

mysql -u root –protocol=tcp -p

password: examly

private string connectionString = "User ID=sa; password=examlyMssql@123; server=localhost**;** Database=appdb; trusted_connection=false; Persist Security Info=False; Encrypt=False";

FRONTEND:

Step 1:

- Use "**cd reactapp**" command to go inside the **reactapp** folder
- Install Node Modules - **"npm install**"

Step 2:

- Write the code inside **src** folder

Step 3:

- Command to run the project "**npm start**"

Step 4:

- To run the test cases, click the "**Run Test Case**" button.

<u>Note:</u>

- Database Name should be **appdb**
- Click PORT 8081 to view the result / output.
- If any error persists while running the app, delete the node modules and reinstall them.

CLOUD DEPLOYMENT

SYSTEM ARCHITECTURE:

The system is divided into two main components: Backend and Frontend. Each component uses various AWS services to handle specific tasks.

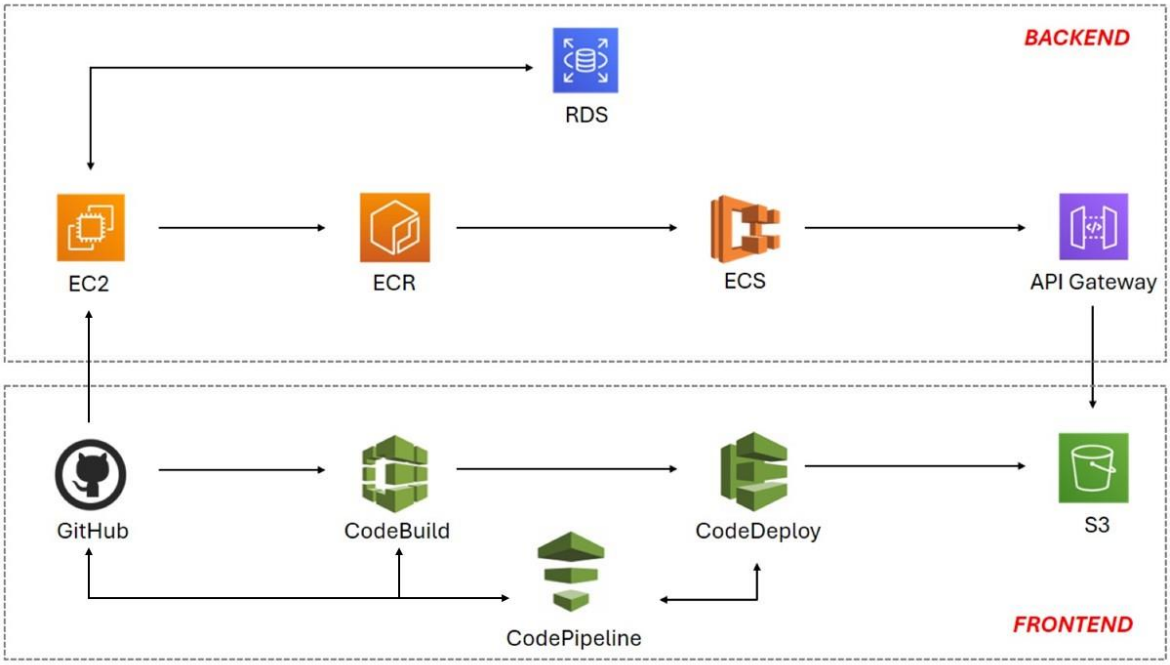
Backend Architecture

- **EC2:** Launch and configure EC2 instance.
- **ECR:** Store Docker images.
- **ECS:** Deploy and manage containers.
- **RDS:** Manage the relational database.
- **API Gateway:** Expose API endpoints.

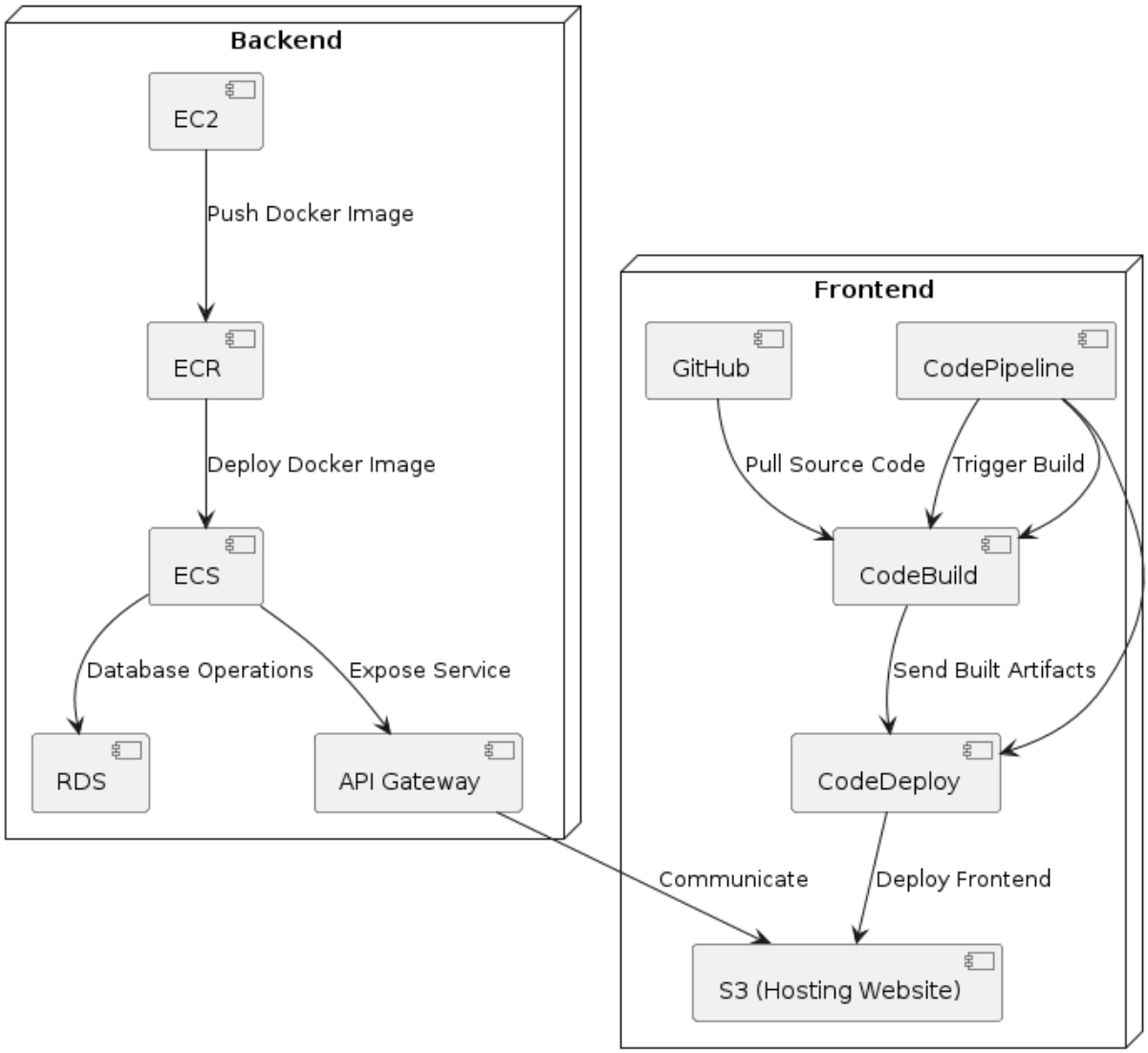
Frontend Architecture

- **GitHub:** Store and manage source code.
- **CodePipeline:** Automate the build and deployment process.
- **CodeBuild:** Build and package the source code.
- **CodeDeploy:** Deploy the built code to S3.
- **S3:** Host the static website.

ARCHITECTURE DIAGRAM:



FLOW DIAGRAM:



<u>SPECIFIC REQUIREMENTS</u>

<u>Backend Services</u>

RDS (Relational Database Service)

- **DB Instance Identifier:** bookfinder-db
- **Region:** us-east-2
- **DB Engine:** MySQL
- **Configuration Steps:**
    - Create a public RDS instance with MySQL engine using free tier template.
    - Configure security groups and network settings.

GitHub

- Navigate to your Github repository where you stored your backend source code.
- Modify the application.properties to update the database URL as the RDS endpoint.

- Set the server address to 0.0.0.0.
- Set the server port to 80.

EC2 (Elastic Compute Cloud)

- **Region:** us-east-2
- **Instance Name:** bmsinstance
- **OS:** Amazon Linux 2023
- **Configuration Steps:**
    - Launch an EC2 instance with the specified OS.
    - Enable "Auto-Assign Public IP".
    - Use your default VPC, subnet and security groups.
    - Connect your EC2 instance to the RDS database.
    - Connect the instance via EC2 instance connect.
    - Install Git on the EC2 instance to clone repositories from GitHub.
    - Clone the necessary GitHub repository containing your backend services code into the EC2 instance.
    - Build the JAR file using Maven and test it in the same EC2 instance.
    - Install Docker and AWS CLI on the instance.
    - Authenticate Docker to push images to ECR.
    - Build and push the Docker image to ECR.

ECR (Elastic Container Registry)

- **Repository Name:** bookfinderrepo
- **Region: us-east-1**
- **Image Tag:** latest
- **Configuration Steps:**
    - Create a new ECR repository.
    - Tag and push Docker images to the repository.

ECS (Elastic Container Service)

- **Cluster Name:** bookfindercluster
- **Region: us-east-2**
- **Task Definition:** bookfindertask
- **Service Name:** bookfinderservice

- **Configuration Steps:**
    - Create an ECS cluster.
    - Define and register the task definition.
    - Create and configure the ECS service for the backend deployment.

API Gateway

- **API Name:** bookrepoapi
- **Region: us-east-2**
- **Main Resource Path:** /api
- **Sub Resource Paths:**
    - /api/books
        - Methods: GET, POST
    - /api/books/{id}
        - Methods: GET, PUT, DELETE
- **Methods Details:**
    - GET /api/books: Retrieve a list of books.
    - POST /api/books: Create a new book
    - GET /api/books/{id}: Retrieve details of the book with the specified ID.
    - PUT /api/books /{id}: Update the book with the specified ID.
    - DELETE /api/books/{id}: Delete the book with the specified ID.
- **Stage Name:** backend
- Enable **CORS** for the API methods.
- **Configuration Steps:**
    - Create a new API named bookrepoapi.
    - Create the main resource path /api.
    - Create sub-resource paths /api/books and /api/books/{id}.
    - Create GET and POST methods for /api/books.
    - Create GET, PUT, and DELETE methods for /api/books/{id}.
    - Enable CORS for all methods.
    - Deploy the API to the stage named backend.
    - Integrate API methods with the ECS service public endpoint.

<u>Frontend Services</u>

GitHub

- Navigate to your GitHub repository where you stored your frontend source code to the repository.
- Add the invoke URL of the API Gateway to the config.js file in the frontend source code.
- Create a buildspec.yml file to define the build instructions and environment settings for CodeBuild.
- Set the discard paths in the buildspec.yml file to 'No' to maintain the directory structure while pushing the build file to the S3 bucket.

S3 (Simple Storage Service)

- **Bucket Region:** us-east-2
- The bucket should have the tag key as **'latest'** and tag value as **'bucket'**.
- Enable **static website hosting**.
- Allow public access for the bucket and its objects.
- Write a bucket policy to allow public access for the objects.
- **Configuration Steps:**
    - Create an S3 bucket.
    - Allow public access for the bucket.
    - Write a bucket policy to allow public access for the objects.
    - Set the Index Document and Error Document as 'index.html'.

CodePipeline

- **Pipeline Name:** bookfinderPipeline
- **Region: us-east-2**
- **Source:** GitHub
- **Stages:** Source, Build, Deploy
- **Configuration Steps:**
    - Create a new pipeline.
    - Configure the source stage to pull from GitHub.
    - Add build and deploy stages using CodeBuild and CodeDeploy.

CodeBuild

- **Project Name:** bookfinderBuild
- **Region: us-east-2**
- **Configuration Steps:**

    - Create a new CodeBuild project.
    - Define the buildspec.yml file in such a way for building your frontend application.

CodeDeploy

- **Deploy to S3:** Host static website
- **Configuration Steps:**
    - Deploy the built artifacts directly to an S3 bucket configured for static website hosting.

Now you can access the website by opening the URL provided by the static website hosting configuration in your S3 bucket.